Backend Development with Node.js

Project Documentation Format

1. Introduction

• Project Title: [Product Inventory Management]

• Team Members: S.RUSHYAN, [Team Leader]

M.SRIKANTH,[Supporter]

M.NEELAKANDAN,[Documendation, Presentation]

M.RAMESH,[Tester& Devlopment]

K.ANANDHAN,[Documendation,Presentation]

2. Project Overview: Product Inventory Management

• Purpose: •To maintain accurate records of all products in the inventory.

•To reduce errors caused by manual inventory management.

•To help businesses easily add, update, delete, and view product details.

•To improve efficiency and save time in inventory handling.

• Features: Add Product – Allows users to add new products with details like product name, ID, price, and quantity.

Update Product Information – Users can edit or update product details such as price, quantity, or supplier information.

Delete Product – Removes products that are no longer available in the inventory.

View Product List – Displays all available products and their details in the system.

Stock Management – Tracks the quantity of products and updates stock when items are added or sold.

Search Product – Helps users quickly find a product using product ID or name.

3. Backend Architecture

• Server Structure: • Client Laye This is the frontend or user interface used by the user.

Examples: Web application or mobile application.

Users perform actions like add product, update stock, delete product, or view inventory.

• Web Server the web server receives requests from the client. It sends the request to the application server for processing.

Examples: Apache, Nginx, or Node.js server.

• API Design: defines how the frontend (user interface) communicates with the backend server to manage inventory data. APIs allow the system to perform operations like adding products, updating stock, and viewing inventory.

• Database Integration: Database Integration in a Product Inventory Management System connects the backend application with the database to store, retrieve, and update inventory data.

4. Technologies Used

• Node.js

Node.js is used in the backend of a Product Inventory Management System to build a fast and scalable server-side application. It allows developers to handle multiple requests efficiently and manage data processing

• Express.js

Express.js is a lightweight web framework used with Node.js to build the backend of the Product Inventory Management System. It helps developers create APIs and manage server operations easily.

• MongoDB Atlas

MongoDB Atlas is a cloud-based database service used to store and manage the data of the Product Inventory Management System. It allows the backend application to store product and inventory information securely on the cloud.

• Mongoose

Mongoose is a library used with Node.js to interact with MongoDB databases easily. In a Product Inventory Management System, it helps manage product data and perform database operations.

• GitHub

GitHub is a platform used to store, manage, and track the source code of the Product Inventory Management project. It helps developers collaborate and maintain different versions of the project.

• Visual Studio Code

Visual Studio Code (VS Code) is a popular code editor used to develop the Product Inventory Management System. It helps developers write, edit, and manage the project's source code efficiently.

5. Setup Instructions

• Prerequisites: Install Node.js

Node.js is required to run the backend server. It also includes npm (Node Package Manager) to install project dependencies.

Create a Database in MongoDB Atlas Used to store product inventory data in the cloud.

A database cluster must be created before connecting the backend. Create an Account on GitHub

Used to download or store the project code

• Installation Steps: install Node.js from the official website.

After installation, verify it using the command:

node -v

npm -v

Install Visual Studio Code

Download and install Visual Studio Code.

Open the project folder in VS Code to edit and run the code

• 1. Clone the project repository

Install Git

Make sure Git is installed on your system.

Open Terminal or Command Prompt

Open the terminal in Visual Studio Code or your system command prompt.

Copy the Repository Link

Go to the project repository on GitHub and copy the repository URL.

Run the Clone Command

Use the following command to download the project:

• 2. Install dependencies using npm install

Open the Project Folder

Open the cloned project folder in Visual Studio Code or any terminal.

Open the Terminal

Use the integrated terminal in VS Code or the system command prompt.

Navigate to the Project Directory

• 3. Configure environment variables

Steps to Configure Environment Variables

Create a .env File

In the project root folder, create a file named .env.

Add Required Variables

Open the .env file and add configuration details such as:

• 4. Start server using npm start or node server.jsSteps to Start the Server

Open the Project Folder

Open the project in Visual Studio Code.

Open the Terminal

Use the integrated terminal in VS Code or the system command prompt.

Navigate to the Project Directory.

6. Folder Structure

• backend/

The backend folder structure organizes the server-side code of the Product Inventory Management system. It helps developers separate database models, API routes, and business logic for easier development and maintenance.

• nnn models/

The Product.js file defines the structure of product data stored in MongoDB Atlas.

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({

  name: {

    type: String,

    required: true

  },
```

```
  price: {
    type: Number,
    required: true
  },
  quantity: {
    type: Number,
    required: true
  },
  category: {
    type: String
  }
});
module.exports = mongoose.model("Product", productSchema);
```

- nnn routes/

Product Routes Example

The productRoutes.js file defines API endpoints for product operations using Express.js.

Java script:

```
const express = require("express");
const router = express.Router();
const productController = require("../controllers/productController");
router.get("/products", productController.getProducts);
router.post("/products", productController.addProduct);
router.put("/products/:id", productController.updateProduct);
router.delete("/products/:id", productController.deleteProduct);
module.exports = router;
```

- nnn controllers/

Product Controller Example

The productController.js file contains functions to manage product inventory using Mongoose.

Java script:

```
const Product = require("../models/Product");

// Get all products

exports.getProducts = async (req, res) => {

  const products = await Product.find();

  res.json(products);

};

// Add a new product

exports.addProduct = async (req, res) => {

  const product = new Product(req.body);

  await product.save();

  res.json(product);

};

// Update product

exports.updateProduct = async (req, res) => {

  const product = await Product.findByIdAndUpdate(req.params.id, req.body, { new: true });

  res.json(product);

};

// Delete product

exports.deleteProduct = async (req, res) => {

  await Product.findByIdAndDelete(req.params.id);

  res.json({ message: "Product deleted successfully" });

};
```

• nnn config/

Database Configuration Example

The db.js file is used to connect the backend application to the database using Mongoose and MongoDB Atlas.

Javascript :

```
const mongoose = require("mongoose");

const connectDB = async () => {

  try {

    await mongoose.connect(process.env.MONGODB_URI);

    console.log("Database connected successfully");

  } catch (error) {

    console.error("Database connection failed:", error);

    process.exit(1);

  }

};

module.exports = connectDB;
```

• nnn server.js

The server is built using Node.js and Express.js, and it connects to the database using Mongoose.

Javascript:

```
const express = require("express");

const mongoose = require("mongoose");

require("dotenv").config();

const productRoutes = require("./routes/productRoutes");

const app = express();

// Middleware

app.use(express.json());

// Routes

app.use("/api", productRoutes);
```

```
// Database connection
mongoose.connect(process.env.MONGODB_URI)
.then(() => console.log("Database connected"))
.catch(err => console.log(err));
// Server start
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(Server running on port ${PORT});
});
```

• nnn package.json

```
{
  "name": "product-inventory-management",
  "version": "1.0.0",
  "description": "Backend for Product Inventory Management System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0",
    "dotenv": "^16.0.0"
  }
}
```

7. API Documentation

• POST /products ®

name: Product name (string, required)

price: Product price (number, required)

quantity: Product quantity (number, required)

image: URL of product image (string, optional)

Response:

Json


```
{
  "_id": "69ae6b7803b34be0d2373781",
  "name": "Notebook",
  "price": 50,
  "quantity": 100,
  "image": "https://via.placeholder.com/150",
  "description": "",
  "userId": "69ad1d9b1480cc2dc8825638",
  "createdAt": "2026-03-09T06:40:56.381Z",
  "updatedAt": "2026-03-09T06:40:56.381Z",
  "__v": 0
}
```


- GET /products ®

Returns an array of product objects:

Json id="4v7yho"


```
[
  {
    "_id": "69ae6b7803b34be0d2373781",
    "name": "Notebook",
    "price": 50,
    "quantity": 100,
    "description": "",
    "userId": "69ad1d9b1480cc2dc8825638",
    "image": "https://via.placeholder.com/150",
```

"createdAt": "2026-03-09T06:40:56.381Z",
"updatedAt": "2026-03-09T06:40:56.381Z",
"__v": 0
},
{
"_id": "69ad9401a5eeb3a4f5f7fc1a",
"name": "Pen",
"price": 10,
"quantity": 50,
"description": "",
"userId": "69ad1d9b1480cc2dc8825638",
"image": "https://via.placeholder.com/150",
"createdAt": "2026-03-08T15:21:37.691Z",
"updatedAt": "2026-03-08T15:21:37.691Z",
"__v": 0
}
]

- PUT /products/:id ®

Json id="q1xz92"

{
"_id": "69ae6b7803b34be0d2373781",
"name": "Notebook Updated",
"price": 60,
"quantity": 120,
"description": "",
"userId": "69ad1d9b1480cc2dc8825638",
"image": "https://via.placeholder.com/150",
"createdAt": "2026-03-09T06:40:56.381Z",

"updatedAt": "2026-03-09T07:00:00.000Z",
"__v": 0
}

- DELETE /products/:I ®

{
"message": "Product deleted successfully"
}

```
fetch(http://localhost:5000/api/v1/products/${productId}, {
  method: "DELETE"
})
.then(res => res.json())
.then(data => console.log(data))
.catch(err => console.error(err));
```

8. Database Schema

- Product Name

{
"_id": "69ae6b7803b34be0d2373781",
"name": "Notebook",
"price": 50,
"quantity": 100,
"description": "A ruled notebook",
"image": "https://via.placeholder.com/150",
"userId": "69ad1d9b1480cc2dc8825638",
"createdAt": "2026-03-09T06:40:56.381Z",
"updatedAt": "2026-03-09T06:40:56.381Z",
"__v": 0
}

- Price


```
{
  "_id": "69ae6b7803b34be0d2373781",
  "name": "Notebook",
  "price": 50,
  "quantity": 100,
  "description": "A ruled notebook",
  "image": "https://via.placeholder.com/150",
  "userId": "69ad1d9b1480cc2dc8825638",
  "createdAt": "2026-03-09T06:40:56.381Z",
  "updatedAt": "2026-03-09T06:40:56.381Z",
  "__v": 0
}
```


- Quantity


```
{
  "_id": "69ae6b7803b34be0d2373781",
  "name": "Notebook",
  "price": 50,
  "quantity": 100,
  "description": "A ruled notebook",
  "image": "https://via.placeholder.com/150",
  "userId": "69ad1d9b1480cc2dc8825638",
  "createdAt": "2026-03-09T06:40:56.381Z",
  "updatedAt": "2026-03-09T06:40:56.381Z",
  "__v": 0
}
```

- Description


```
{
  "_id": "69ae6b7803b34be0d2373781",
  "name": "Notebook",
  "price": 50,
  "quantity": 100,
  "description": "A ruled notebook for school use",
  "image": "https://via.placeholder.com/150",
  "userId": "69ad1d9b1480cc2dc8825638",
  "createdAt": "2026-03-09T06:40:56.381Z",
  "updatedAt": "2026-03-09T06:40:56.381Z",
  "__v": 0
}
```


- Created Date


```
{
  "_id": "69ae6b7803b34be0d2373781",
  "name": "Notebook",
  "price": 50,
  "quantity": 100,
  "description": "A ruled notebook for school use",
  "image": "https://via.placeholder.com/150",
  "userId": "69ad1d9b1480cc2dc8825638",
  "createdAt": "2026-03-09T06:40:56.381Z",
  "updatedAt": "2026-03-09T06:40:56.381Z",
  "__v": 0
}
```


9. Running the Application

• Run command: npm start

Install Dependencies

Make sure you have Node.js installed.

Open VS Code terminal or your system terminal and navigate to your backend folder:

Bash id="r1y2b6"

cd path/to/your/backend/folder

• Server runs on: http://localhost:5000

•Install Dependencies

Make sure Node.js is installed.

Open terminal and go to your backend folder:

Bash

cd path/to/your/backend

npm install

• Start the Backend Server

Run the backend using:

Bash

npm start

You should see output like:

Server running on port 5000

MongoDB connected

10. Testing

• Testing Tools:

•• Testing Tools

Postman – For testing API endpoints like GET, POST, PUT, DELETE

Browser – For testing the frontend webpage functionality

•• Testing API Endpoints with Postman

a) GET /products

Purpose: Fetch all products

Steps:

Open Postman

Create a new GET request

URL: http://localhost:5000/api/v1/products

Click Send

Expected Result:

Response returns an array of products with _id, name, price, quantity, etc.

- Test operations:

  •• Create Product (POST /products)

  Purpose: Add a new product to the inventory.

  Steps:

  Open Postman or your frontend webpage.

  Fill product details:

  Name, Price, Quantity, Image URL (optional)

  Click Send (Postman) or Add Product (webpage).

  Expected Result:

  Product is added to MongoDB.

  Frontend shows the new product in the grid.

  Response includes _id, createdAt, and other details.

  •• {

  "name": "Notebook",

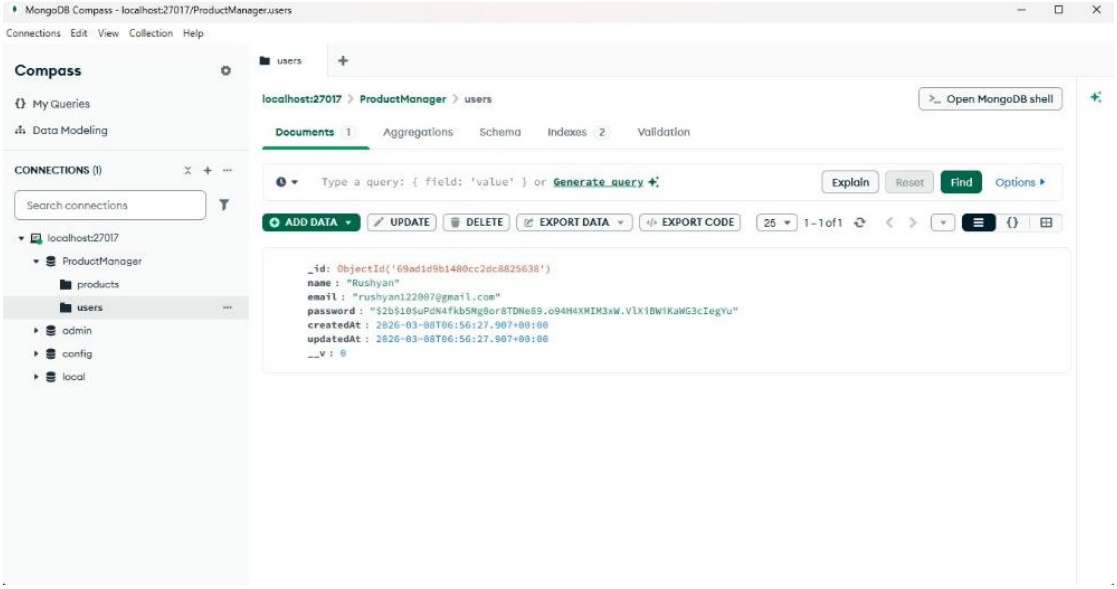
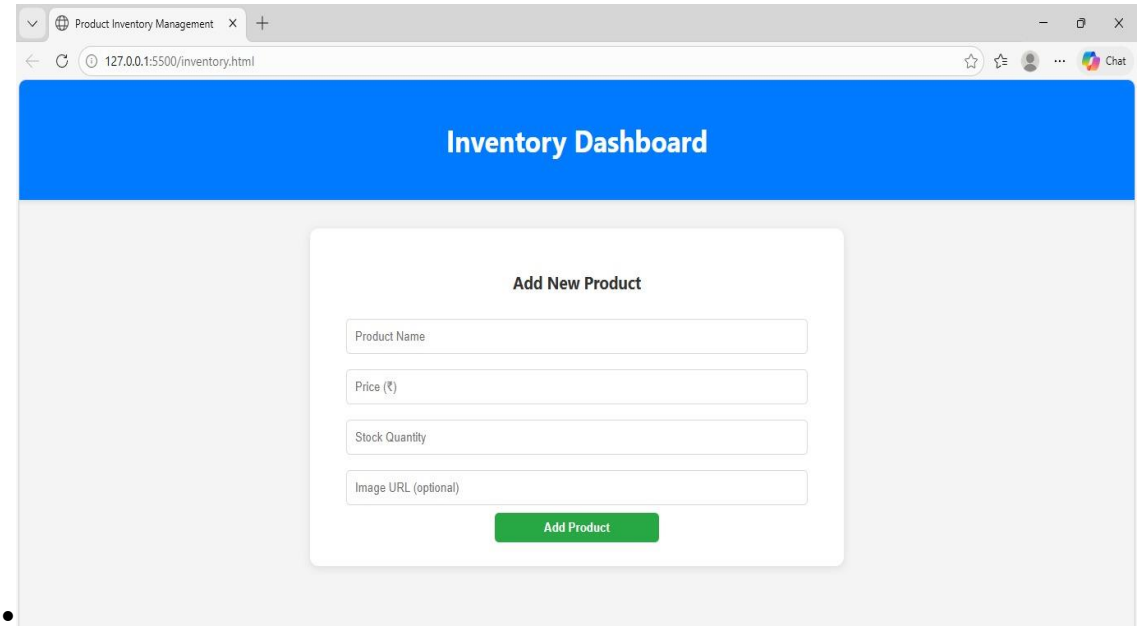
  "price": 50,

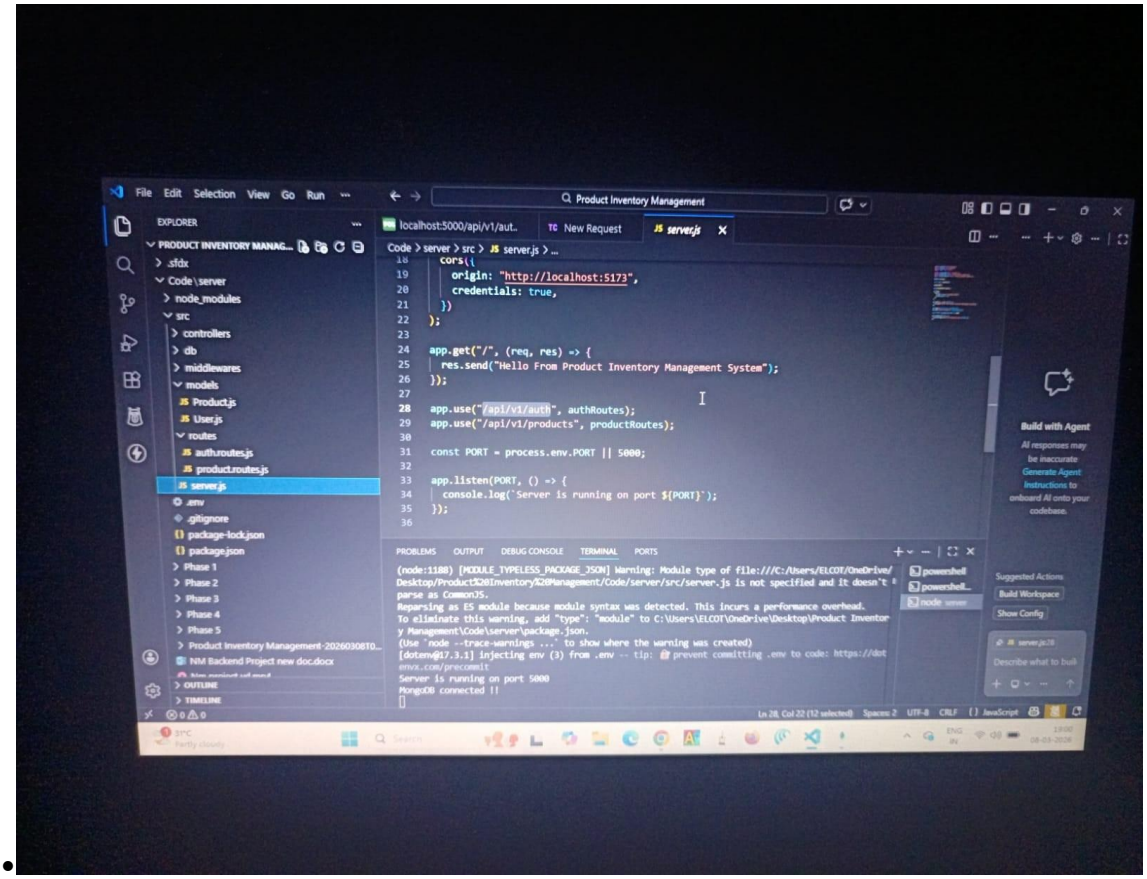
  "quantity": 100,

  "image": "https://via.placeholder.com/150"

  }

11. Screenshots / Demo

- 
MongoDB Compass - localhost:27017/ProductManager.users
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
ProductManager
products
users
admin
config
local
users
localhost:27017 > ProductManager > users
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
25
1-1of1
_id: ObjectId('69ad1d9b1480cc2dc8825638')
name : "Rushyan"
email : "rushyan122007@gmail.com"
password : "$2b$10$uPdN4fkb5Mg0or8TDNe69.o94H4XMIM3xW.VlX1BW1KaWG3cIegYu"
createdAt : 2026-03-08T06:56:27.907+00:00
updatedAt : 2026-03-08T06:56:27.907+00:00
__v : 0
- 
Product Inventory Management
127.0.0.1:5500/inventory.html
Chat
Inventory Dashboard
Add New Product
Product Name
Price (₹)
Stock Quantity
Image URL (optional)
Add Product



12. Known Issues

- Missing product images use a placeholder.
- Negative or invalid values for price/quantity are not validated.
- Concurrent updates may overwrite previous changes.
- No authentication; any user can modify products.
- Backend errors may not display clearly on frontend.
- Large datasets may slow page load (no pagination).
- No search, filter, or sort features.
- Minor layout issues on older browsers.

13. Future Enhancements

- Add user authentication and role-based access to secure CRUD operations.
- Implement input validation to prevent negative or invalid product data.
- Add search, filter, and sort options for products.
- Implement pagination to handle large datasets efficiently.

- Enable image upload instead of URL input for products.
- Add report generation (stock, sales, low inventory).
- Improve error handling and notifications on frontend.
- Make the application responsive for mobile and tablet devices.

14. Conclusion

• Developing the Product Inventory Management System helped in understanding backend development, API creation, and database management.

Key learnings include:

- How to design and implement CRUD operations for products.
- How to connect a backend to MongoDB to store and manage product data.
- How to handle requests and responses between frontend and backend.
- Understanding the importance of data validation, error handling, and proper API design.